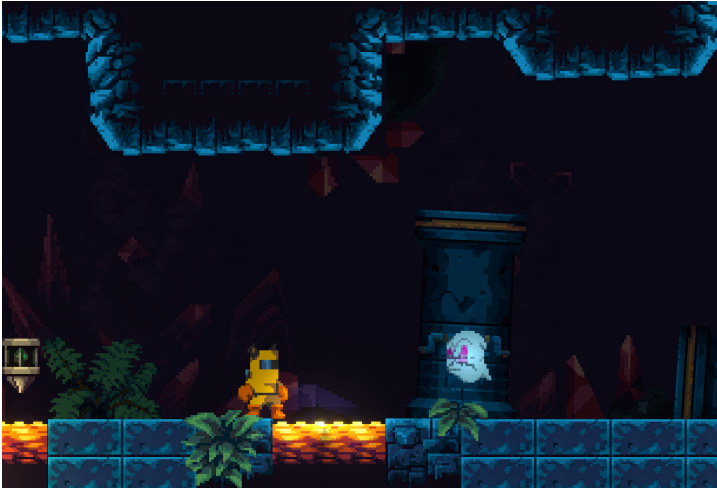


Projet en classe : La caverne de la mort

Dans les prochains cours (16 à 23), nous allons réaliser un jeu de type “platformer” où le joueur contrôle un personnage qui doit traverser un niveau et éviter des ennemis et des obstacles dangereux. Voici un [exemple jouable](#) de ça qu’on va créer en classe.



1.1 Cours 17 - Déplacer le jouer avec la physique

On va implémenter ce déplacement en deux étapes : déplacement simple et avec des fonctionnalités extras.

1.1.1 Étape 1 - Déplacement de base

1. Ajouter un **Rigidbody2D** et un **BoxCollider2D** à l'objet Personnage.
2. Dans `Start()`, on récupère une référence `rb` au **Rigidbody2D** avec `GetComponent<>()`.
3. Créer un nouveau script `DeplacementPlatformer.cs`.

1.1.1.1 Configuration de contrôle

1. Importer la bibliothèque `UnityEngine.InputSystem`.
2. Définir deux variables publiques `InputAction` nommées `entreeMarche` et `entreeSaut`.
3. On prépare les `InputActions` dans `OnEnable()` et `OnDisable()`.
4. Dans l'**Inspector**, on ajoute des bindings : un positif/négatif pour `actionMarche` et un simple pour `actionSaut`.
5. Pour le mouvement et l'input, on va synchroniser les inputs avec la physique. Pour faire ça, on va dans **Edit > Project-Settings > Input System Package > Settings** et on change **Update Mode > Process Events in Fixed Update**. Pour plus de détails, voir [la documentation](#).
6. Dans notre script, on va créer une fonction `FixedUpdate()` et, dans cette fonction, deux variables `float` locales : `axeMarche` avec la valeur de notre `actionMarche` et `intensiteMarche` avec la valeur absolue de cet axe (`Mathf.Abs(axeMarche)`).

1.1.1.2 Appliquer la logique de la marche On veut avoir du mouvement de marche avec des accélérations pour donner un style plus fluide aux déplacements.

1. Créer des champs publics float pour `accelerationMarche`, `vitesseXMax` et `vitesseXActuelle`.
2. On multiplie `axeMarche` par `accelerationMarche` et on somme au `rb.linearVelocityX`.
3. On limite la valeur de `rb.linearVelocityX` avec un `Mathf.Clamp()` et la `vitesseXMax`.
4. On garde la valeur de `rb.linearVelocityX` dans la variable `vitesseXActuelle` parce qu'elle sera utile pour gérer les animations et le visuel du personnage.

1.1.1.3 Appliquer la logique du saut Pour le saut, on va commencer avec une implémentation qui ne limite pas l'activation du saut et où son hauteur est toujours la même.

1. Changer le Gravity Scale du Rigidbody2D à 5f.
2. Créer des champs publics float pour `accelerationSaut` et `vitesseYMax`.
3. Si `actionSaut.WasPressedThisFrame()`, on va commencer le saut, donc le `tempsSaut = 0f`.
 1. On applique l'impulsion vers le haut avec la méthode `rb.AddForce(Vector2.up * impulsionSaut, ForceMode2D.Impulse)`. Le `Vector2.up` définit la direction et `impulsionSaut` l'intensité de la force. L'argument `ForceMode2D.Impulse` applique cette accélération de façon immédiate.
 2. Pour éviter des déplacements extrêmes, on limite la valeur de `rb.linearVelocityY` avec un `Mathf.Clamp()` et la `vitesseYMax`.

1.1.1.4 Configuration exemple -pour l'étape 1 Voici quelques valeurs intéressantes pour la configuration du composant :

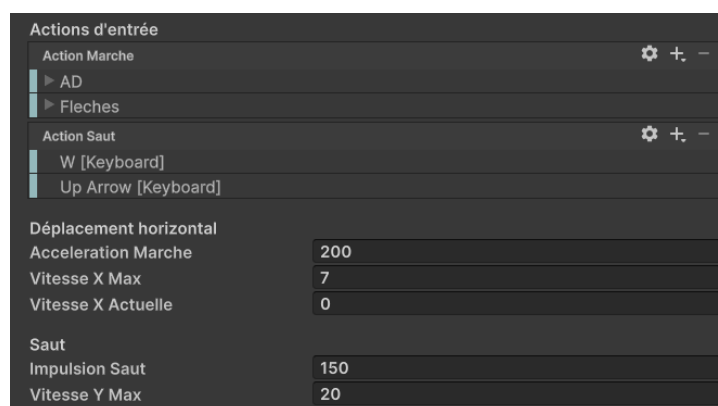


Figure 1 – alt text

1.1.2 Étape 2 - Détection du sol et saut variable

1.1.2.1 Modifier la logique de la marche On veut avoir du mouvement de marche avec des accélérations pour donner un style plus fluide aux déplacements.

1. Changer le Linear Damping du Rigidbody2D a 0f.
2. Créer un champs publiques float pour ralentissementMarche.
3. Pour **ralentir la marche** :
4. Si on n'as pas d'intensité de marche (`intensiteMarche <= 0.01f`), on ralentisse la vitesse X du rb avec une interpolation vers 0f (`Mathf.Lerp(rb.linearVelocityX, 0, ralentissementMarche * Time.fixedDeltaTime)`).

1.1.2.2 Modifier la logique du saut Pour le saut, on veut le modifier pour qu'il soit activé seulement quand le personnage est au sol. On veut aussi que l'impulsion du saut a une durée maximale : ça nous permet d'avoir un saut d'hauteur variable selon le temps qu'on enfonce la touche, en lieu d'une hauteur fixe.

1. Créer des champs publics float pour tempsSaut, dureeImpulsionSaut, vitesseYMax, un champs LayerMask calquesSol et un champs bool estAuSol.
2. Pour faire la **détection du sol** :
 1. On va tester si le personnage est au sol avec la méthode `Physics2D.Raycast(rb.position, Vector2.down, 1f, calquesSol)` qui lance un rayon d'un mètre de longueur à partir de son pivot vers en bas. Les objets hors les calquesSol vont être ignorés.
 2. On garde le résultat dans une variable locale `RaycastHit2D hit`.
 1. Si `hit.collider` est nul, le rayon n'a pas touché d'objets dans les calques du sol.
 2. Si `hit.collider` n'est pas nul, le rayon a touché le sol.
 3. On garde le résultat de ces conditions dans la variable `estAuSol`.
3. Pour créer **la hauteur variable**, on va ajouter deux if et modifier le if déjà existant :
 1. Si `actionSaut.WasPressedThisFrame()` && `estAuSol`, on va commencer le saut, donc le tempsSaut = 0f.
 2. Dans le if du `actionSaut.IsPressed()`, on augmente le tempsSaut pour montrer que plus de temps enfoncé c'est passé.
 3. Si `actionSaut.WasReleasedThisFrame()`, le bouton est relâché, donc on change le tempsSaut à l'infini pour éviter que l'impulsion soit appliquée.

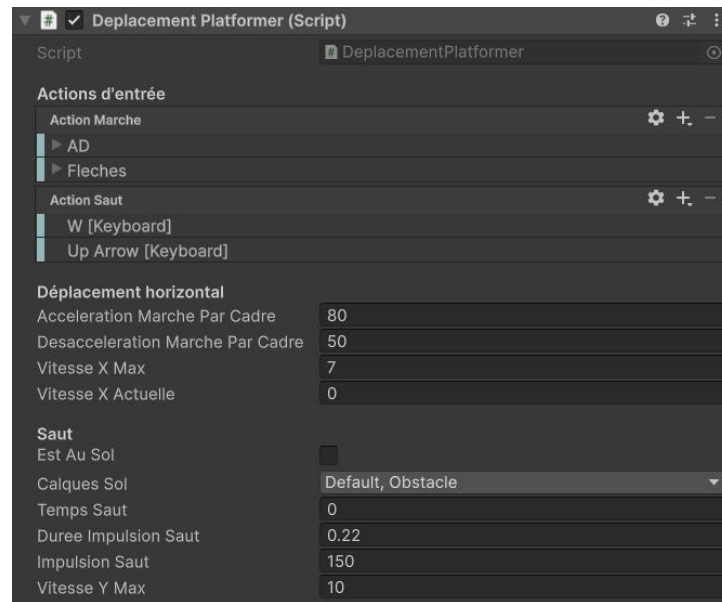


Figure 2 – Config étape 2

1.1.2.3 Configuration exemple pour l'étape 2

1.2 Cours 18 - Contrôle de caméra avec Cinemachine

1.2.1 Configuration de base : 2D Camera

1. Si le package **Cinemachine** n'est pas installé, aller sur *Window > PackageManager > Unity Registry* et l'installer.
2. On ajoute un composant **Cinemachine Brain** à la **Main Camera** de la scène.
3. On crée un nouvel objet dans la scène avec *GameObject > Cinemachine > Targeted Cameras > 2D Camera*. Nommer cet objet **CinemachineCamera**.
4. Dans le nouvel objet **CinemachineCamera**, on change la propriété **Tracking Target** à notre objet de Personnage.
5. On change la propriété **Lens** à 5 pour répliquer la taille de la caméra défaut.

1.2.2 Composition dynamique

Le composant **Cinemachine Position Composer** permet d'encadrer l'objet cible (**Tracking Target**) dynamiquement, avec des **zones mortes** (où la caméra ne suit pas la cible), **anticipation** (Lookahead, quand la caméra suit une estimation de la position future de la cible) et des **limites de cadre**.

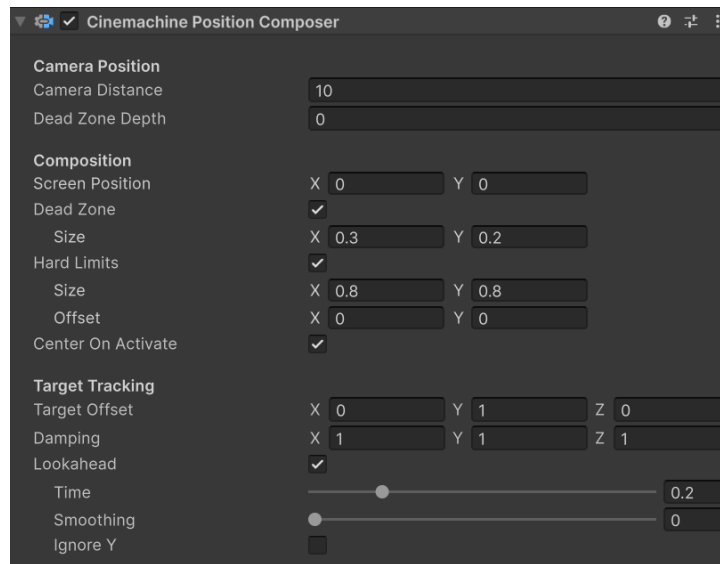


Figure 3 – Configuration du Position Composer

1.2.3 Limites de niveau pour la caméra

On peut aussi définir des limites sur l'espace général de la scène. Les étapes pour configurer des limites sont :

1. On clique sur le bouton *Add Extension* de notre **CinechineCamera** et on ajoute un composant **Cinemachine Confiner2D**.
2. On ajoute un nouvel objet vide dans la scène nommé **LimitesCameraNiveau** avec un **BoxCollider2D**. Éditer la forme du collider pour délimiter l'espace de la caméra. Activer l'option *IsTrigger*.
3. On connecte cet objet **LimitesCameraNiveau** à la propriété *Bounding Shape 2D* du **Confiner2D**.

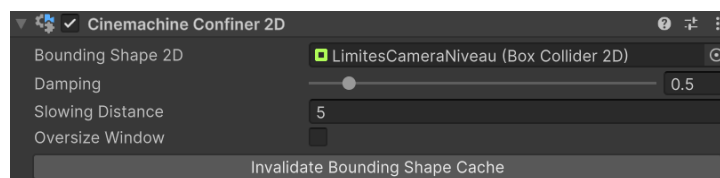


Figure 4 – Configuration du Confiner2D

1.3 Cours 19 - Gestion d'animations multiples avec Animator pt.1

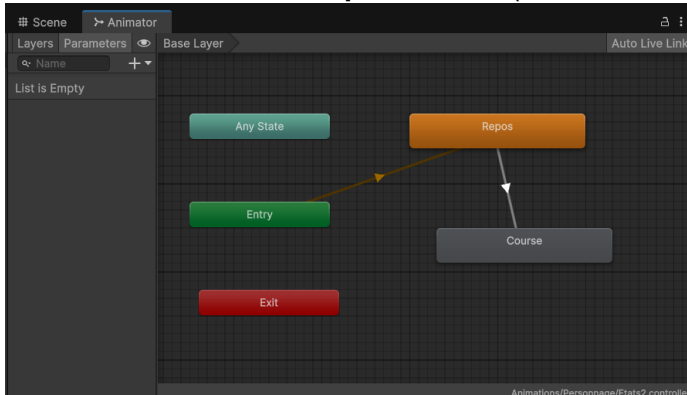
1.3.1 Préparation des animations : course, repos

1. On utilise le **Sprite Editor** pour découper et définir les pivots des sprites *player-idle* et *player-run*.
2. Avec le panneau **Animation** et l'objet *Visuel* de notre *Personnage* sélectionné, on va créer deux **animations image par image**, une pour le repos et une autre pour la course. Les deux doivent être en boucle (**Animation Clip > Loop Time actif**).
3. La création de ces animations va aussi créer un composant **Animator** à l'objet *Visuel* et un fichier du type **Animator**.

Controller. On va double-cliquer le fichier du type Animator Controller pour explorer notre **machine d'état qui va gérer les animations multiples.**

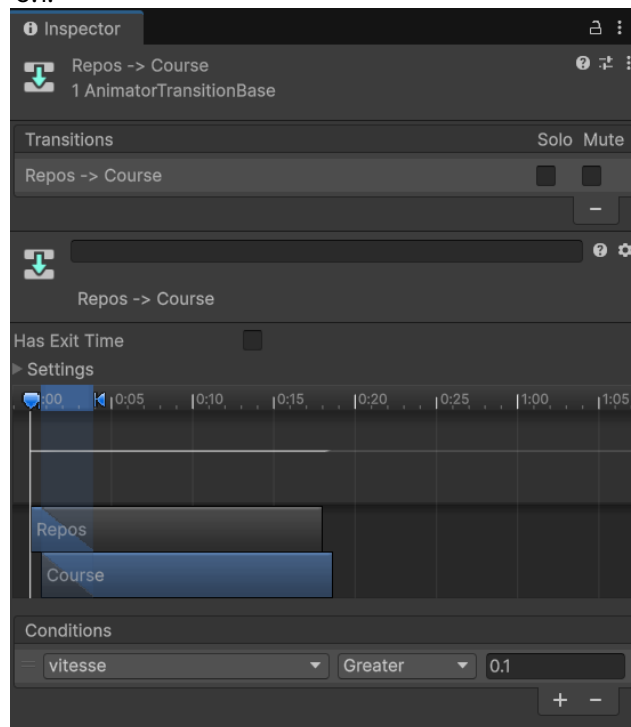
1.3.2 Configuration des états, paramètres et transitions

1. On va définir l'animation de **repos** comme notre animation défaut (*Bouton droit sur l'état > Set Default Layer State*).
2. On crée une transition de **repos à course** (*Bouton droit > Make Transition*).



3. On va aussi créer une transition de retour **course à repos**.
4. Après sélectionner les flèches de chaque transition, on désactive l'option *HasExitTime* des transitions.
5. On retourne au panneau Animator. Dans l'onglet Parameters, on va ajouter un nouveau paramètre du type float nommé **vitesse**.
6. Dans les transitions, on va créer des **conditions** en utilisant ce paramètre :

1. Transition *Repos > Course* : vitesse Greater 0.1.



2. Transition *Course > Repos* : vitesse Less 0.1.

1.3.3 Connexion avec notre logique de jeu (scripts)

1. Pour envoyer les informations de nos scripts vers l'Animator, on utilise les fonctions `SetFloat()`, `SetBool()` et `SetTrigger()` de la composante **Animator**.
2. Dans un nouveau script `Personnage.cs` ajouté à notre objet *Personnage*, on va prendre des références à l'**Animator** de l'objet *Visuel* et à notre composant **DeplacementPlatformer** :

```
Animator anim;
DeplacementPlatformer deplacement;

void Start(){
    anim = GetComponentInChildren<Animator>();
    deplacement = GetComponent<DeplacementPlatformer>();
}
```

3. On va récupérer la vitesse horizontale du personnage, calculer son intensité, c.à.d. la valeur absolue `Mathf.Abs()`, et garder dans une variable float `intensiteVitesseX`.
4. Après, on va envoyer cette information au paramètre `vitesse` de notre Animator, en utilisant la méthode `SetFloat()`.

```
void Update(){
    float intensiteVitesseX = 0f;
    intensiteVitesseX = Mathf.Abs(deplacement.vitesseX);
    anim.SetFloat("vitesse", intensiteVitesseX);
}
```

1.4 Cours 20 - Gestion d'animations multiples avec Animator pt.2

À venir.

1.5 Cours 21 - Instanciation de projectiles

À venir.

1.6 Cours 22 - Gestion d'ennemis et listes

À venir.

1.7 Cours 23 - Post processing et lumières 2D

À venir.